

存储管理是数据库管理系统（DBMS）的重要职责之一。了解关系数据库的数据组织与存储管理，能帮助我们更好地理解关系数据库查询优化的原理，也有助于我们在应用开发中更好地进行数据库物理设计。

9.1 基本知识点

本章讲解基于磁盘的数据库的组织与存储。

- ① 需要了解的：数据库的逻辑组织方式与物理组织方式。
- ② 需要牢固掌握的：掌握记录如何在块中组织存储以及如何组织关系表的存放；深入理解索引的作用，包括顺序表索引、辅助索引、B+树索引、哈希索引和位图索引等。
- ③ 需要深入了解的：B+树索引和哈希索引的组织方式与查找方式。
- ④ 难点：理解数据库逻辑组织与物理组织之间的对应关系。

9.2 习题解答和解析

1. 试分析每个数据库对象对应一个操作系统文件与整个数据库对应一个或若干个文件，这两种存储关系数据库的策略各有什么优缺点？

【解析】请阅读《概论》第9章9.1.1小节“数据库的逻辑组织方式与物理组织方式”相关内容。

每个数据库对象对应一个操作系统文件，这在本质上是将存储管理交由操作系统完成。其优势在于方便DBMS实现，其缺点在于不能利用数据库的特点来优化存储管理。而整个数据库对应一个或若干个文件，这在本质上是由DBMS自己负责存储管理。其优缺点与前者正好相反。

2. 假设Course表以定长记录方式存储。请描述Course表的记录存储在以下情况中一条记录占多少字节。

- ① 字段可以在任何字节处开始。
- ② 字段必须在4的倍数的字节处开始。
- ③ 字段必须在8的倍数的字节处开始。

【解析】Course 表的定义，可以参阅《概论》第3章例3.6 建立一个“课程”表 Course，即

```
CREATE TABLE Course
(Cno CHAR(5) PRIMARY KEY,      /*列级完整性约束，Cno 是主码*/
Cname VARCHAR(40) NOT NULL,   /*列级完整性约束，Cname 不能取空值*/
Ccredit SMALLINT,
Cpno CHAR(5),                  /* Cpno 的含义是 Cno 课程的直接先修课 */
FOREIGN KEY (Cpno) REFERENCES Course(Cno)
/*表级完整性约束，Cpno 是外码，被参照表是 Course，被参照列是 Cno */
);
```

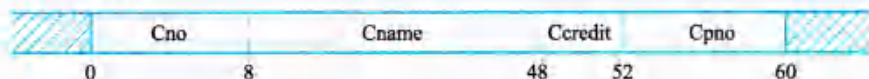
① 字段可以在任何字节处开始。

一条记录占 52 字节。



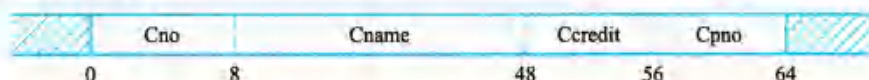
② 字段必须在 4 的倍数的字节处开始。

一条记录占 60 字节。



③ 字段必须在 8 的倍数的字节处开始。

一条记录占 64 字节。



3. 试述关系表有哪些组织方式，并分析各自的优缺点。

【解析】请阅读《概论》第9章9.1.4 小节“关系表的组织”相关内容。

常见的关系表组织方式包括堆存储、顺序存储、多表聚簇存储、B+树存储和哈希存储等。

① 堆存储：表中的一条记录可以存储在该表的任何块中，没有顺序要求。堆存储的优点是可以高效地进行插入操作，在插入记录时只需找到合适的空闲空间即可。其缺点是搜索、更新、删除操作代价较高，需要扫描全表以找到所有满足条件的记录。

② 顺序存储：一个表中的各条记录按照指定的属性或属性组的取值大小顺序存储。顺序存储方式的优点是可以高效地处理按排序属性（组）进行查询的请求。其缺点是具有较高的维护代价，在插入、更新和删除记录时，可能需要在块内或块间迁移已有记录来保持记录顺序。

③ 多表聚簇存储：不同表的记录聚簇存储在同一组块中。聚簇存储相当于对不同表进行预连接。其优点是可以减少连接操作的开销。其缺点一是降低了单表查询的效率，因为同一个关系表的记录会分散在更多的块里；二是增加了更新操作的代价。在关系表中插入新记录时，

为保证不同表之间的记录聚簇存放,可能需要频繁地迁移记录来腾出空间。

④ B+树存储:以B+树索引的形式确定记录存储在哪个数据块中。B+树存储方式的优点是能在保持较高的记录访问效率的同时降低数据维护开销。其缺点是按不等于某个值的条件进行搜索时效率很低。

⑤ 哈希存储:用哈希函数计算表中指定属性的哈希值,以此确定相应记录存储在哪个块中。哈希存储方式的优点是可以通过哈希函数,可以将记录的存储位置与其指定属性的取值之间建立一个对应关系,因此在按哈希属性进行等值条件查询时,可以通过计算哈希属性值快速定位到记录的位置。其缺点是不适合数据分布严重倾斜的情况,而且也不能快速完成一切非等值查询,如范围查询、按大于某个值进行的查询等。

4. 试述数据库索引机制的优点。

【解析】请阅读《概论》第9章9.2节“索引结构”相关内容。

索引是一种针对关系表的类似于图书目录的附加结构,可用来提高查询效率。索引机制的主要优点有:

① 一个表的索引块数量通常比数据块数量少得多,因而搜索起来就会比较快。

② 索引通常采用一些易于检索的数据结构,可以使用高效的方法在索引中快速查找。

③ 对于经常访问的数据库表,如果其索引文件足够小,则可以让其长久地驻留在内存缓冲区中,从而减少I/O操作。

5. 试述稠密索引和稀疏索引的优缺点。

【解析】请阅读《概论》第9章9.2.1小节“顺序表索引”相关内容。

① 稠密索引的索引块中存放每条记录的索引属性值,以及指向相应记录的指针。稠密索引的优点在于仅通过扫描索引就可以准确定位记录。其缺点为当关系表的记录数比较多时,索引也会比较大,这将导致在索引中查找可能会花费较多的时间;与此同时,对基本表进行增删改时,稠密索引的维护代价相对较高。

② 稀疏索引的每个索引项存放每个物理块的第一条记录的索引属性值,以及指向该物理块的指针。稀疏索引的优点为索引尺寸较小且维护成本较低。当对基本表进行增删改时,只要修改的记录属性不是存储块第一条记录的索引属性,稀疏索引就不需要维护。其缺点为查询效率比稠密索引低一些,因为这种索引不能精确定位记录,所以仍需在数据块中进行顺序查找。

6. 分别描述图9-12的稠密索引、图9-13的稀疏索引、图9-14的多级索引、图9-20的B-树索引、图9-23的哈希索引,是如何对Student表进行如下查询的。

【解析】请读者阅读《概论》第9章9.2节“索引结构”相关内容。注意,请从查找算法的角度来描述如何利用不同的索引结构对Student表进行查询。

① 查询学号为20180013的记录。

稠密索引:依次读入索引块。由于稠密索引是有序索引,因此,可以在索引块中利用顺序查找或二分查找法,查找Sno为20180013的索引项。在第3个索引块中找到该索引项,根据索引项的指针读取相应的存储块,并在存储块中取出Sno为20180013的记录。

稀疏索引：依次读入索引块。首先，在索引项中利用顺序查找或二分查找法，查找属性值小于或等于 20180013 的最大索引项，找到在第 2 个索引块中有值为 20180013 的索引项。然后，根据该索引项中的指针读取相应的存储块。最后，在存储块中顺序搜索，查找并取出 Sno 属性值为 20180013 的记录。

多级索引：首先，在二级索引中按稀疏索引的查找方法定位 20180013 索引项在一级索引中索引块的位置，依次读取二级索引块，在索引块中利用顺序查找或二分查找法，查找 Sno 属性值小于或等于 20180013 的最大索引项，即第一个索引块中的 20180013。然后，根据该索引项中的指针读取一级索引块，在这个索引块中查找 20180013 索引项。最后，根据该索引项的指针读取存储块，在存储块中顺序查找并取出 Sno 属性值为 20180013 的记录。

B+树索引：首先，在根结点中进行判断，由于 $20180013 > 20180008$ ，因此沿右子树继续搜索。然后，在中间结点，由于 $20180013 > 20180012$ ，因此沿 20180012 右侧指针继续向下搜索。最后，在叶结点中找到 20180013 后，用其左侧指针从数据库中取出 Sno 属性值为 20180013 的记录。

哈希索引：首先，利用哈希函数计算出 20180013 索引项所在的桶号，为 1 号桶。然后，在 1 号桶中查找 20180013 索引项，根据索引项的指针在存储块中取出 Sno 属性值为 20180013 的记录。

② 查询学号为 20180014 的记录。

稠密索引：依次读入索引块。由于稠密索引是有序索引，因此，可以在索引块中利用顺序查找或二分查找法，查找 Sno 属性值为 20180014 的索引项。在第 3 个索引块中找到该索引项，根据索引项的指针读取相应的存储块，并在存储块中取出 Sno 为 20180014 的记录。

稀疏索引：依次读入索引块。首先，在索引项中利用顺序查找或二分查找法，查找属性值小于或等于 20180014 的最大索引项，找到在第 2 个索引块中有值为 20180013 的索引项。然后，根据该索引项中的指针，读取相应的存储块。最后，在存储块中顺序搜索，查找并取出 Sno 属性值为 20180014 的记录。

多级索引：首先，在二级索引中按稀疏索引的查找方法，定位 20180014 索引项在一级索引中索引块的位置，依次读取二级索引块，在索引块中利用顺序查找或二分查找法，查找 Sno 属性值小于或等于 20180014 的最大索引项，即第一个索引块中的 20180013。然后，根据该索引项中的指针读取一级索引块，在这个索引块中查找 20180014 索引项。最后，根据该索引项的指针读取存储块，在存储块中顺序查找并取出 Sno 属性值为 20180014 的记录。

B+树索引：首先，在根结点中进行判断，由于 $20180014 > 20180008$ ，因此沿右子树继续搜索。然后，在中间结点，由于 $20180014 > 20180012$ ，因此沿 20180012 右侧指针继续向下搜索。最后，在叶结点中找到 20180014 后，用其左侧指针从数据库中取出 Sno 属性值为 20180014 的记录。

哈希索引：首先，利用哈希函数计算出 20180014 索引项所在的桶号，为 2 号桶。然后，在 2 号桶中查找 20180014 索引项，根据索引项的指针，在存储块中取出 Sno 属性值为 20180014

的记录。

③ 查询学号为 20180016 的记录。

稠密索引：依次读入索引块。由于稠密索引是有序索引，且最后一个索引块中的最后一个索引项是 20180015 小于 20180016，因此判断数据库中不存在 Sno 属性值为 20180016 的记录。

稀疏索引：依次读入索引块。首先，在索引项中利用顺序查找或二分查找法，查找属性值小于或等于 20180016 的最大索引项，找到在第 2 个索引块中有值为 20180015 的索引项。然后，根据该索引项中的指针，读取相应的存储块。最后，在存储块中顺序搜索，由于记录是顺序存储的，并且该存储块的最后一条记录为 20180015，小于 20180016，可知 Sno 属性值为 20180016 的记录在数据库中不存在。

多级索引：依次读取二级索引块，在索引块中利用顺序查找或二分查找法，查找 Sno 属性值小于或等于 20180016 的最大索引项，找到属性值为 20180013 的索引项。然后，根据该索引项的指针读取一级索引块，在这个索引块中查找 20180016 索引项。由于一级索引是稠密索引，且当前索引块中最大索引项是 20180015，可知 Sno 属性值为 20180016 的记录在数据库中不存在。

B+树索引：首先在根结点中进行判断，由于 $20180016 > 20180008$ ，因此沿右子树继续搜索。然后，在中间结点，由于 $20180016 > 20180012$ ，因此沿 20180012 右侧指针继续向下搜索。最后，在叶结点中未找到 20180016 索引项，所以该记录在关系表中不存在。

哈希索引：首先，利用哈希函数计算出 20180016 索引项所在的桶号，为 1 号桶。在 1 号桶中未找到 20180016 索引项，所以 20180016 记录在关系表中不存在。

④ 查询学号大于或等于 20180009 的记录。

稠密索引：依次读入索引块，由于稠密索引是有序索引，因此可以在索引块中利用顺序查找或二分查找法，查找 Sno 属性值为 20180009 的索引项。根据该索引项中的指针读入存储块，取出满足条件的所有记录。由于文件是顺序存放的，所以后面所有记录的 Sno 均大于或等于 20180009，因此依次读入之后的所有存储块并取出相应记录。

稀疏索引：依次读入索引块，找到小于或等于 20180009 的最大索引项，即 20180009。根据该索引项中的指针读入存储块，取出满足条件的所有记录。由于文件是顺序存放的，所以后面所有记录的 Sno 均大于或等于 20180009，因此依次读入之后的所有存储块并取出相应记录。

多级索引：首先，读入二级索引（稀疏索引）的索引块，从中找到小于或等于 20180009 的最大索引项，即 20180007。然后，根据二级索引项的指针，读取相应的一级索引（稠密索引）索引块，找到索引块中 Sno 属性值大于或等于 20180009 的索引项，再根据索引项的指针读取存储块，从存储块中取出第一条满足条件的记录。由于关系表是顺序存放的，可知该记录后的所有记录均满足条件，因此依次遍历并取出该存储块中的记录。遍历完当前存储块后，依次读取其后的存储块及其中的记录，直到关系表的最后。

B+树索引：首先，从根节点开始沿相应的父子结点指针逐层向下搜索，直到在叶结点中找到属性值为 20180009 的索引项，可知其后所有叶结点中的索引项属性值均大于或等于 20180009。根据当前叶结点中的指针读入存储块，读取存储块中满足条件的记录。遍历完当前叶结点后，沿指向下一叶结点的指针继续遍历，直到最后一个叶结点。

哈希索引：哈希索引本身并不能保证记录的顺序，因此不能利用哈希索引进行范围查询。通过依次读取每个存储块，遍历每条记录并取出所有 $Sno \geq 20180009$ 的记录。

7. 描述当依次用下列方式更新 Student 表中的记录时，图 9.12 的稠密索引、图 9.13 的稀疏索引、图 9.14 的多级索引、图 9.16 的辅助索引、图 9.20 的 B+树索引、图 9.23 的哈希索引分别是如何进行维护的。

① 插入记录(20180016, 张婧, 女, 2002-1-2, 信息安全)。

稠密索引：依次读入索引块，查找 Sno 小于或等于 20180016 的最大索引项，即 20180015。根据索引项指针读取存储块，找到 Sno 小于或等于 20180016 的最大记录(其 Sno 为 20180015)。该存储块中仍有空闲空间，因此将存储插入当前存储块的第一个空位中，然后在索引块中增加对应的索引项。

20180013	20180013	李昱	男	2000-10-1	信息管理与信息系统
20180014	20180014	沈录	男	1999-12-2	信息管理与信息系统
20180015	20180015	邓瑶	女	2001-2-22	信息安全
20180016	20180016	张婧	女	2002-1-2	信息安全

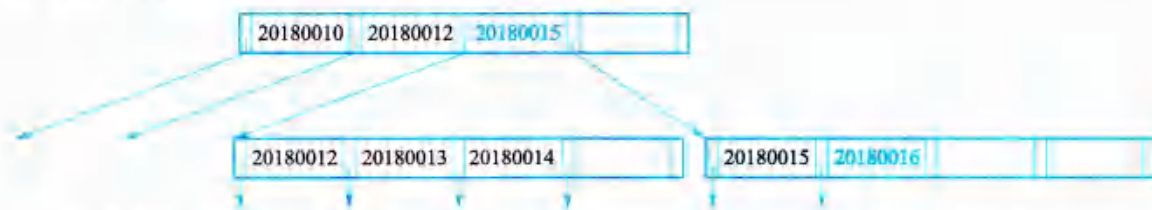
稀疏索引：根据索引，查找 Sno 小于或等于 20180016 的最大索引项，即 20180015。根据索引项指针读取存储块，该存储块中仍有空闲空间，因此直接将这条记录插入存储块。由于新插入的记录不在存储块的开头，因此无须更新索引。

20180013	20180015	20180015	邓瑶	女	2001-2-22	信息安全
		20180016	张婧	女	2002-1-2	信息安全

多级索引：根据多级索引，查找 Sno 小于或等于 20180016 的最大索引项，即 20180015。根据索引项指针读取存储块，该存储块中仍有空闲空间，因此直接将这条记录插入存储块，然后在一级索引块中增加对应的索引项。由于新插入的记录不在存储块的开头，因此无须更新第二级索引。



B+树索引：首先在关系表中插入这条记录，根据 B+树索引找到插入索引项的位置。由于当前结点中属性值 key 的个数等于 4，已经达到最大充满度，因此需要将这个叶结点分裂成两个结点（即增加一个新的叶结点），其中一个结点存放 Sno 为 20180012、20180013 以及 20180014 的 3 个索引项，新增加的叶结点存放 Sno 为 20180015 和 20180016 的 2 个索引项，并且把 20180016 属性值左侧的指针指向新插入记录所在的存储块位置。然后，在 B+树的中间结点中插入属性值 key 20180015，并令指针指向新增加的叶结点。



哈希索引：首先在关系表中插入这条记录，根据哈希函数计算插入索引项的位置 ($16\%3=1$) 为 1 号桶。1 号桶中仍有空闲空间，因此将索引项插入 1 号桶，并且令索引项的指针指向存储块中新插入记录的位置。



② 删除学号为 20180004 的记录。

稠密索引：通过稠密索引查找算法，在存储块中找到 Sno 为 20180004 的记录。先将该记录在存储块中删除，然后在索引块中删除相应的索引项，并把该索引块中位于 20180004 后的索

引项朝着起始方向迁移。



稀疏索引：通过稀疏索引查找算法，在存储块中找到 Sno 为 20180004 的记录，删除该记录。由于该记录不是所在存储块的第 1 条数据，因此无须更新稀疏索引的索引项。

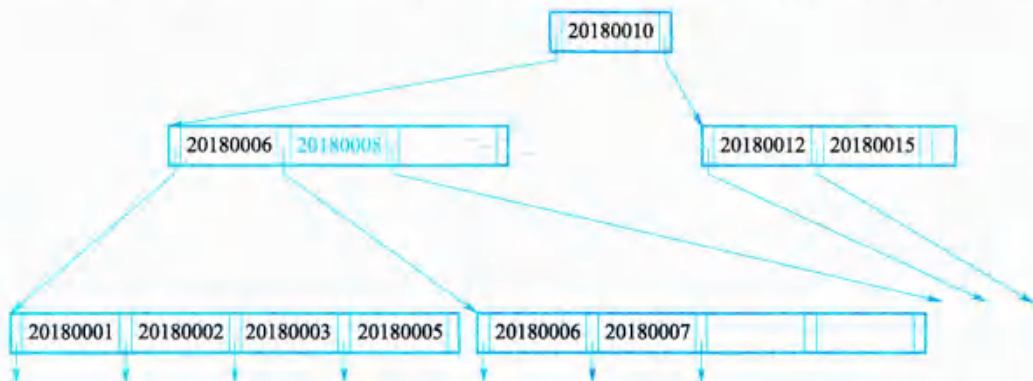


多级索引：通过多级索引查找算法，在存储块中找到 Sno 为 20180004 的记录，删除该记录。在一级索引（稠密索引）中找到该记录对应的索引项并删除，然后将同一索引块中那些位于其之后的索引项朝着起始方向迁移。由于删除的索引项不是该一级索引块中的第 1 条索引项，因此二级索引（稀疏索引）无须更新。

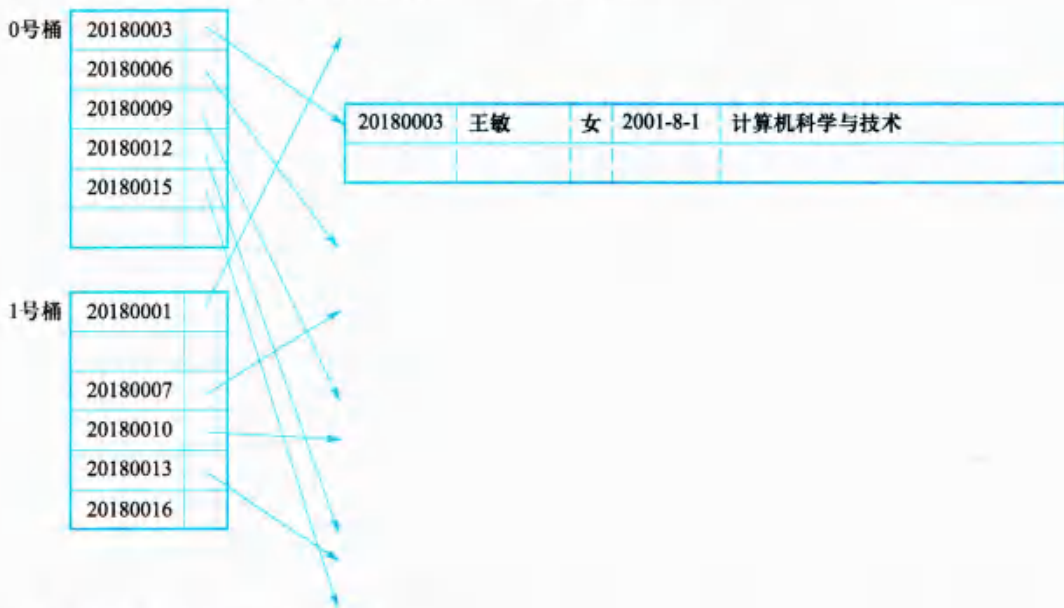


B+树索引：根据 B+树索引，找到并删除叶结点中 20180004 的索引项。因删除 20180004 索引项后当前叶结点只有 1 个索引项，小于最小充满度，而其左兄弟结点中有 3 个索引项，未达到最大充满度，所以将属性值为 20180005 的索引项并入左叶结点，然后删除父结点中的

20180004 及其右指针。父结点删除 20180004 后只剩下一个属性值，因此需要与其右兄弟结点重新分配属性值，并根据重分配后的属性值修改根结点的属性值。更新后的 B+树如下图所示：



哈希索引：根据哈希函数计算 20180004 所在的桶号，为 1 号桶，然后到 1 号桶中找到 20180004 索引项，删除该索引项及其所指向的存储块中的记录。



③ 删除学号为 20180005 的记录。

稠密索引：通过稠密索引查找算法，在存储块中找到 Sno 为 20180005 的记录，先将该记录在存储块中删除，视空闲空间管理方法的不同，位于该记录后的数据可以向数据块起始方向迁移，也可以维持现状；然后在索引块中删除相应的索引项，并把该索引块中位于 20180005 后的索引项朝着起始方向迁移。



稀疏索引：通过稀疏索引查找算法，在存储块中找到 Sno 为 20180005 的记录，删除该记录。由于该记录是所在数据块的第 1 条数据，因此需要将该记录后的数据向数据块起始方向迁移，并需要更新稀疏索引的索引项，将属性值为 20180005 的索引项更新为 20180006。

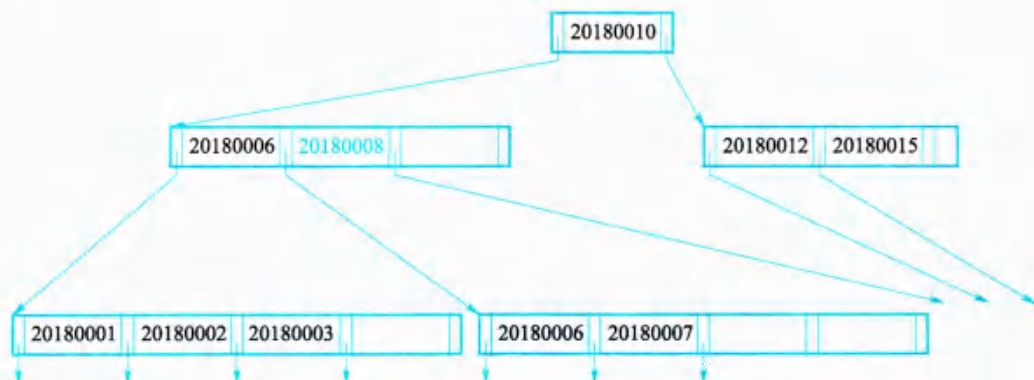


多级索引：通过多级索引查找算法，在存储块中找到 Sno 为 20180005 的记录，删除该记录。在一级索引（稠密索引）中找到该记录对应的索引项并删除，然后将同一索引块中那些位于其之后的索引项朝着起始方向迁移。由于删除的索引项不是该一级索引块中的第 1 条索引项，因此二级索引（稀疏索引）无须更新。

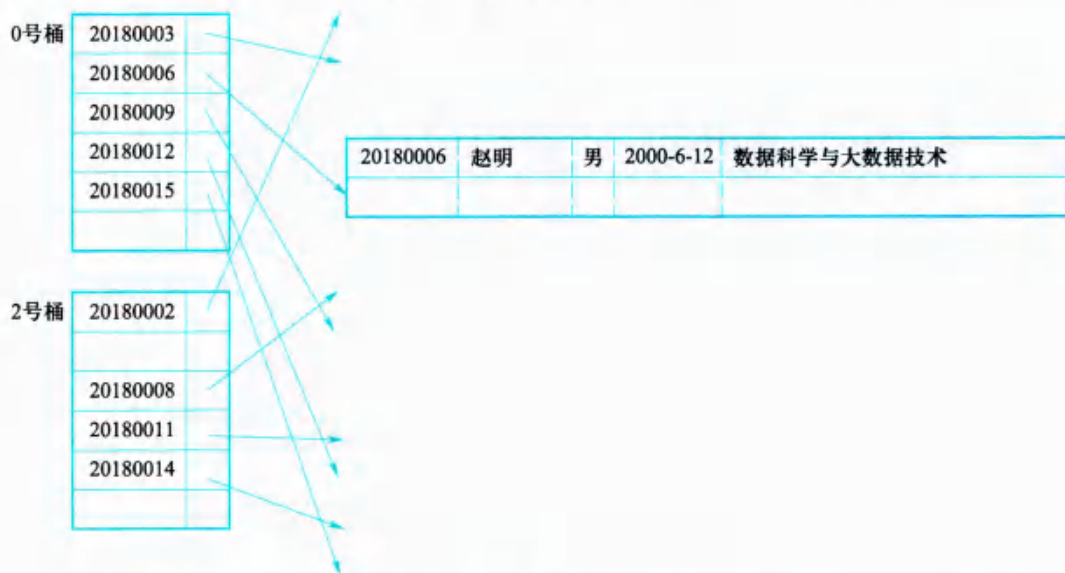


B+树索引：根据 B+树索引，找到叶结点中 20180005 的索引项并删除。因为删除该索引项

后当前叶结点有 3 个索引项, 大于最小充满度, 所以直接删除该索引项及相应指针即可, 如下图所示:



哈希索引: 根据哈希函数计算 20180005 所在的桶号, 为 2 号桶。到 2 号桶中找到 20180005 索引项, 删除该索引项及其指向的存储块中的记录。



④ 将学号为 20180008 的记录修改为计算机科学与技术专业。

答: 根据各个索引的查找算法, 找到 20180008 记录, 将这条记录从存储块中取出后, 修改其专业为计算机科学与技术。由于索引是建立在学号上的, 因此无须更新索引。

第 10 章

关系查询处理和查询优化

第 10 章讲解了关系数据库管理系统 (RDBMS) 查询处理和查询优化的基本概念、方法和

技术。
查询优化是 RDBMS 内部实现的技术,对于一般用户而言是隐蔽的。用户不必了解 RDBMS 如何针对查询语句进行优化,然而对于 DBA 和应用开发人员来说,“性能调优 (performance tuning)”是其职责之一,因此需要掌握查询优化的概念和 RDBMS 的内部优化技术,进而为在数据库应用开发中利用查询优化技术提高查询效率和系统性能打下基础。

10.1 基本知识点

本章首先介绍 RDBMS 的查询处理步骤和实现查询操作的算法,然后深入讨论 RDBMS 查询优化的基本概念和方法,包括基于关系代数等价变换规则的优化方法、基于启发式规则的存取路径优化和基于代价估算的优化等方法。

查询处理是 RDBMS 的核心,它是 RDBMS 语言处理中最重要、最复杂的部分。

为了提高关系数据库系统的执行效率,RDBMS 必须进行查询优化。由于关系查询语言 (例如 SQL) 具有较高的语义层次,使得 RDBMS 可以进行查询优化,这就形成了 RDBMS 查询优化的必要性和可能性。

① 需要了解的:了解查询处理的基本步骤;了解查询分析、查询检查、查询优化和查询执行。

② 需要牢固掌握的:掌握什么是 RDBMS 的查询优化,查询优化的方法,特别是基于启发式规则的优化和基于代价估算的优化。

③ 需要举一反三的:能够画出一个查询的语法树以及优化后的语法树。

④ 难点:掌握查询优化算法,包括代数优化算法和物理优化算法。

10.2 习题解答和解析

1. 试述查询优化在关系数据库管理系统中的重要性和可能性。

【解析】请阅读《概论》第 10 章 10.2.1 小节“查询优化概述”相关内容。